

Learning-Rate Scheduling, Worked Out by Hand

Step, exponential, cosine and warmup schedules tabulated

A complete numerical walk-through of the four learning-rate schedules the course shows side by side. We fix one base learning rate, $\eta_0 = 0.1$, and from the closed-form rule of each schedule we compute the *exact* learning rate at every epoch — by hand, then re-derived from `torch.optim.lr_scheduler` so the arithmetic is self-checking. Nothing is hand-waved: every number below was produced by the NumPy/PyTorch program in Appendix A and agrees with PyTorch to nine decimals.

What this document does. It takes the single quantity an optimiser actually reads each step — the learning rate η_t — and shows how a *scheduler* rewrites it over training. For *StepLR*, *ExponentialLR*, *CosineAnnealingLR*, and a *linear-warmup + cosine* schedule we write the formula, plug in $\eta_0 = 0.1$, and tabulate η_t epoch by epoch.

How to read this. Each schedule gets its own worked section with a formula, a boxed closed form, and a table of η versus epoch. The coloured boxes say *why* each shape is useful and where the one classic bug lives (stepping the scheduler at the wrong cadence). Read this right after the optimiser topic: the scheduler does not change the update rule, only the scalar η that multiplies it.

Concepts covered, in order: why schedule at all · StepLR as a staircase $\eta_0 \gamma^{\lfloor t/s \rfloor}$ · ExponentialLR as smooth geometric decay $\eta_0 \gamma^t$ · CosineAnnealing $\eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos \frac{\pi t}{T})$ · linear warmup then cosine · per-epoch vs per-batch `scheduler.step()` · choosing a schedule for the task.

1 The setup: one base rate, four shapes

The optimiser update is always $\theta \leftarrow \theta - \eta_t g_t$ (SGD) or some preconditioned variant (Adam). A **learning-rate schedule** leaves that rule untouched and only changes the scalar η_t from one step to the next. So the entire topic reduces to a single one-dimensional question: *given the base rate η_0 , what is η_t at epoch t ?*

We fix the shared constant for every schedule below:

Quantity	Symbol	Value here
Base / max learning rate	$\eta_0 = \eta_{\max}$	0.1
Step decay factor	γ (StepLR)	0.1
Step size	s	3 epochs
Exponential decay factor	γ (Exp)	0.9
Cosine period	T	10 epochs
Cosine floor	$\eta_{\min}$	0
Warmup length	w	3 epochs

Intuition

A high learning rate early on lets the optimiser take big strides and *explore* the loss landscape quickly; a low rate late lets it *converge* into a minimum without overshooting. Every schedule below is one way to interpolate between "big and exploratory" and "small and careful". StepLR drops in discrete cliffs; Exponential bleeds down geometrically; Cosine glides smoothly to (near) zero; warmup adds a short ramp *up* at the very start before any decay.

2 Step 1 — StepLR: a decaying staircase

StepLR holds the rate constant for s epochs, then multiplies it by γ . After k completed blocks of s epochs the rate has been multiplied k times:

$$\boxed{\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}} \quad \text{with } \eta_0 = 0.1, \gamma = 0.1, s = 3.$$

The exponent is $\lfloor t/s \rfloor$, the number of full step-blocks elapsed. Plugging in epoch by epoch:

$$\begin{aligned} t = 0, 1, 2 : \quad \lfloor t/3 \rfloor &= 0, \quad \eta = 0.1 \cdot 0.1^0 = 0.1 \\ t = 3, 4, 5 : \quad \lfloor t/3 \rfloor &= 1, \quad \eta = 0.1 \cdot 0.1^1 = 0.01 \\ t = 6, 7, 8 : \quad \lfloor t/3 \rfloor &= 2, \quad \eta = 0.1 \cdot 0.1^2 = 0.001 \end{aligned}$$

The rate is flat inside each block and falls by a factor of 10 at every boundary $t = 3, 6, \dots$:

epoch t	0	1	2	3	4	5	6	7	8
$\lfloor t/3 \rfloor$	0	0	0	1	1	1	2	2	2
η_t	0.1	0.1	0.1	0.01	0.01	0.01	0.001	0.001	0.001

Watch out

PyTorch applies the drop *after* the s -th `step()`, so epoch 3 is the first epoch at the lower rate, not epoch 2. The exponent is $\lfloor t/s \rfloor$ with t counting completed scheduler steps. Off-by-one here is common: print `opt.param_groups[0]['lr']` at the top of each epoch (before stepping) to see the value the optimiser will actually use that epoch.

3 Step 2 — ExponentialLR: smooth geometric decay

ExponentialLR multiplies by γ *every* epoch instead of every s epochs, so the staircase becomes a smooth geometric curve:

$$\boxed{\eta_t = \eta_0 \gamma^t} \quad \text{with } \eta_0 = 0.1, \gamma = 0.9.$$

Each epoch retains a fraction $\gamma = 0.9$ of the previous rate — a steady 10% drop per epoch. Evaluating $0.1 \cdot 0.9^t$:

$$\begin{aligned} \eta_0 &= 0.1 \cdot 0.9^0 = 0.100, & \eta_1 &= 0.1 \cdot 0.9^1 = 0.090, & \eta_2 &= 0.1 \cdot 0.9^2 = 0.081, \\ \eta_3 &= 0.1 \cdot 0.9^3 = 0.073, & \eta_4 &= 0.1 \cdot 0.9^4 = 0.066, & \eta_5 &= 0.1 \cdot 0.9^5 = 0.059. \end{aligned}$$

epoch t	0	1	2	3	4	5	6	7	8
0.9^t	1.000	0.900	0.810	0.729	0.656	0.590	0.531	0.478	0.430
η_t	0.100	0.090	0.081	0.073	0.066	0.059	0.053	0.048	0.043

Mini-example — this operation in isolation

Geometric decay never reaches zero; it only halves at a fixed pace. The half-life is $t_{1/2} = \ln(0.5)/\ln(\gamma)$. For $\gamma = 0.9$ that is $\ln 0.5 / \ln 0.9 \approx 6.58$ epochs, which matches the table: η falls from 0.100 to about 0.053 (just over half) by epoch 6. Smaller γ = shorter half-life = faster decay.

4 Step 3 — CosineAnnealing: a smooth glide to the floor

CosineAnnealingLR sweeps the rate from $\eta_{\max}$ down to $\eta_{\min}$ following half a cosine wave over a period of T epochs:

$$\boxed{\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos \frac{\pi t}{T}\right)} \quad T = 10, \eta_{\max} = 0.1, \eta_{\min} = 0.$$

With $\eta_{\min} = 0$ this simplifies to $\eta_t = 0.05(1 + \cos \frac{\pi t}{T})$. The phase $\frac{\pi t}{T}$ runs from 0 (at $t = 0$, $\cos = 1$, full rate) to π (at $t = T$, $\cos = -1$, zero rate). Worked at the requested epochs:

$$\begin{aligned} t = 0 : \frac{\pi \cdot 0}{10} &= 0, & \cos 0 &= 1, & \eta &= 0.05(1 + 1) = 0.100 \\ t = 2 : \frac{\pi \cdot 2}{10} &= 0.2\pi, & \cos &= 0.809, & \eta &= 0.05(1 + 0.809) = 0.090 \\ t = 5 : \frac{\pi \cdot 5}{10} &= 0.5\pi, & \cos &= 0.000, & \eta &= 0.05(1 + 0) = 0.050 \\ t = 8 : \frac{\pi \cdot 8}{10} &= 0.8\pi, & \cos &= -0.809, & \eta &= 0.05(1 - 0.809) = 0.010 \\ t = 10 : \frac{\pi \cdot 10}{10} &= \pi, & \cos &= -1, & \eta &= 0.05(1 - 1) = 0.000 \end{aligned}$$

epoch t	0	2	5	8	10
phase $\pi t/T$	0	0.2π	0.5π	0.8π	π
$\cos(\pi t/T)$	1.000	0.809	0.000	-0.809	-1.000
η_t	0.100	0.090	0.050	0.010	0.000

Note the symmetry about the midpoint $t = 5$: the curve is exactly $\eta_{\max}/2 = 0.05$ there, and the values at $t = 2$ and $t = 8$ (0.090 and 0.010) are mirror images about 0.05 because $\cos(0.8\pi) = -\cos(0.2\pi)$.

Intuition

Cosine spends a long time near the top (the curve is flat where $\cos$ peaks) so the optimiser explores at high rate for many epochs, then accelerates the decay through the middle, then flattens again near zero for a gentle, fine-tuning landing. That "slow, fast, slow" shape is why cosine annealing so often beats a linear ramp for training-from-scratch.

5 Step 4 — Linear warmup, then cosine decay

Adaptive optimisers (Adam/AdamW) and large-batch training are unstable at the very first steps, when the running statistics are still garbage. **Warmup** fixes this by ramping the rate *up* linearly from a small value over the first w epochs, *then* handing off to a decay schedule. With $w = 3$ the warmup phase is

$$\eta_t = \eta_{\max} \frac{t+1}{w} \quad (t < w), \quad \eta_{\max} = 0.1, \quad w = 3,$$

which gives $\eta_0 = 0.1 \cdot \frac{1}{3} = 0.033$, $\eta_1 = 0.1 \cdot \frac{2}{3} = 0.067$, and $\eta_2 = 0.1 \cdot \frac{3}{3} = 0.100$ — a straight line up to the peak. From $t = w$ onward we run a cosine decay over the remaining $T - w = 7$ epochs, with progress $p = \frac{t-w}{T-w}$:

$$\eta_t = \frac{1}{2} \eta_{\max} \left(1 + \cos(\pi p) \right), \quad p = \frac{t-3}{7} \quad (t \geq w).$$

For example $t = 5$ gives $p = \frac{2}{7}$, $\cos(\frac{2}{7}\pi) = 0.623$, so $\eta = 0.05(1 + 0.623) = 0.081$. The full schedule:

epoch t	0	1	2	3	4	5	6	7	8	9	10
phase	up	up	up	dn	dn	dn	dn	dn	dn	dn	dn
η_t	0.033	0.067	0.100	0.100	0.095	0.081	0.061	0.039	0.019	0.005	0.000

The peak $\eta = 0.100$ is reached at $t = 2$ (end of warmup) and held into $t = 3$ (start of cosine, where $p = 0$ and $\cos 0 = 1$), after which the cosine glides it down to 0.

When this is the right choice

Use **cosine annealing** as a strong default for training a model *from scratch*: the smooth glide to a near-zero floor reliably lands in a good minimum. Add **linear warmup** for transformers, very large batch sizes, or any Adam/LAMB run — the ramp keeps the first few updates from diverging while the optimiser's moments stabilise. Reach for **StepLR** when you are *fine-tuning in stages* and want explicit, reproducible rate drops at known epochs; **ExponentialLR** when you want a single knob (γ) for a steady, gentle decay.

6 Step 5 — The one bug: stepping at the wrong cadence

A scheduler advances its internal epoch counter every time you call `scheduler.step()`. Whether the tables above describe *epochs* or *batches* depends entirely on *where* you put that call.

Watch out

Call `scheduler.step()` **after** `optimizer.step()`, and exactly **once per intended period**. Put it in the epoch loop $\Rightarrow$ one decay per epoch (the tables above). Put it in the batch loop $\Rightarrow$ one decay per batch, so with 100 batches/epoch a $T = 10$ cosine empties in 10 batches, not 10 epochs. The classic crash is *double-stepping*: calling `step()` in both loops, which decays twice as fast and silently ruins training. Pick one cadence and set the scheduler's $T/s/\gamma$ to match it.

The course's reference pattern makes the order explicit: optimiser first, scheduler second, then read back the rate.

Call order inside the loop	Effect
<code>loss.backward()</code>	accumulate gradients
<code>optimizer.step()</code>	apply the update with the <i>current</i> η_t
<code>scheduler.step()</code>	advance the counter $\Rightarrow$ set η_{t+1}

7 The four schedules, at a glance

Schedule	Rule for η_t	Shape	η at $t=0$	η at last t
StepLR	$\eta_0 \gamma^{\lfloor t/s \rfloor}$	staircase down	0.100	0.001
ExponentialLR	$\eta_0 \gamma^t$	smooth geometric	0.100	0.043
Cosine	$\frac{1}{2}\eta_{\max}(1 + \cos \frac{\pi t}{T})$	slow-fast-slow	0.100	0.000
Warmup+cosine	ramp up, then cosine	up then down	0.033	0.000

The "last t " column is epoch 8 for StepLR/Exp and epoch 10 for the two cosine schedules. Every value in this table is read directly from the worked sections above.

8 Equation cheat-sheet

StepLR	$\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}$ (drop $\times \gamma$ every s epochs)
ExponentialLR	$\eta_t = \eta_0 \gamma^t$ (drop $\times \gamma$ every epoch)
CosineAnnealing	$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos \frac{\pi t}{T})$
Linear warmup ($t < w$)	$\eta_t = \eta_{\max} (t + 1)/w$
Warmup half-life (Exp)	$t_{1/2} = \ln(0.5)/\ln(\gamma)$
Cosine midpoint	$\eta_{T/2} = \frac{1}{2}(\eta_{\max} + \eta_{\min})$
Correct cadence	<code>optimizer.step()</code> then <code>scheduler.step()</code> , once per period

Appendix A Reference implementation

Every number above is reproducible from the following program. The closed forms are evaluated in NumPy and then cross-checked against the real `torch.optim.lr_scheduler` objects; they agree to nine decimal places.

```
import numpy as np, torch, torch.nn as nn
r3 = lambda x: round(float(x), 3)
base = 0.1

# (1) StepLR:  $\eta_0 * \gamma^{\lfloor t/s \rfloor}$       gamma=0.1, s=3
step = [base * 0.1**(t//3) for t in range(9)]
# (2) ExponentialLR:  $\eta_0 * \gamma^t$       gamma=0.9
exp = [base * 0.9**t for t in range(9)]
# (3) CosineAnnealing:  $\eta_{\min} + 0.5 * (\eta_{\max} - \eta_{\min}) * (1 + \cos(\pi t/T))$ 
T, lo, hi = 10, 0.0, 0.1
cos = {t: lo + 0.5*(hi-lo)*(1+np.cos(np.pi*t/T)) for t in [0,2,5,8,10]}
# (4) linear warmup(w=3) then cosine over the remaining T-w epochs
w = 3
def warmcos(t):
    if t < w:
        return hi * (t+1) / w
    p = (t - w) / (T - w)
    return lo + 0.5*(hi-lo)*(1 + np.cos(np.pi*p))
warm = [warmcos(t) for t in range(T+1)]

print("StepLR :", [r3(x) for x in step])
print("Exp    :", [r3(x) for x in exp])
print("Cosine :", {t: r3(v) for t, v in cos.items()})
print("Warmup :", [r3(x) for x in warm])

# ---- cross-check against real torch schedulers ----
def run(make, n):
    opt = torch.optim.SGD(nn.Linear(1, 1).parameters(), lr=base)
    sch = make(opt)
    out = []
    for _ in range(n):
        out.append(opt.param_groups[0]['lr']) # read BEFORE stepping
        opt.step()
        sch.step()                             # scheduler AFTER optimizer
    return out

L = torch.optim.lr_scheduler
t_step = run(lambda o: L.StepLR(o, step_size=3, gamma=0.1), 9)
t_exp  = run(lambda o: L.ExponentialLR(o, gamma=0.9), 9)
t_cos  = run(lambda o: L.CosineAnnealingLR(o, T_max=10, eta_min=0.0), 11)

assert all(abs(a-b) < 1e-9 for a, b in zip(t_step, step))
assert all(abs(a-b) < 1e-9 for a, b in zip(t_exp, exp))
for t in [0, 2, 5, 8, 10]:
    assert abs(t_cos[t] - cos[t]) < 1e-9
print("all torch checks passed")
# StepLR : [0.1, 0.1, 0.1, 0.01, 0.01, 0.01, 0.001, 0.001, 0.001]
# Exp    : [0.1, 0.09, 0.081, 0.073, 0.066, 0.059, 0.053, 0.048, 0.043]
# Cosine : {0: 0.1, 2: 0.09, 5: 0.05, 8: 0.01, 10: 0.0}
# Warmup : [0.033, 0.067, 0.1, 0.1, 0.095, 0.081, 0.061, 0.039, 0.019, 0.005, 0.0]
```

— end of document —